

# Análise dos Impactos na Memória Cache

Pedro Henrique França\*

## 1. Programa de Simulação

O simulador de memória cache foi implementado na linguagem C e possibilita a configuração completa dos parâmetros da cache: política de escrita (*write-through* ou *write-back*), tamanho da linha, número de linhas, associatividade, política de substituição (LRU ou aleatória) e tempos de acesso. O programa recebe como entrada um arquivo texto contendo endereços de 32 bits em hexadecimal e a operação correspondente (R ou W).

### 1.1 Estruturas de dados

Quatro estruturas principais compõem o simulador:

- **LinhaCache** — representa uma linha da cache, contendo os campos *valido*, *rotulo* (*tag*), *dirty* e *lru* (contador para o algoritmo LRU).
- **Config** — armazena todos os parâmetros de configuração recebidos pela linha de comando.
- **Cache** — estrutura central que mantém o vetor de linhas alocado dinamicamente, o número de conjuntos, a associatividade e os valores pré-calculados de bits de *offset* e de índice.
- **Estatísticas** — acumula contadores de leituras, escritas, acertos e acessos à memória principal.

### 1.2 Algoritmos e funções principais

**Decomposição do endereço.** Cada endereço de 32 bits é dividido em três campos: *offset* (bits menos significativos, determinado por  $\log_2(\text{tamanho\_linha})$ ), índice do conjunto ( $\log_2(\text{número de conjuntos})$ ) e rótulo (bits restantes). As funções *extraí\_indice* e *extraí\_rotulo* realizam essa extração usando deslocamentos de bits e máscaras.

**Política de substituição.** A função *escolhe\_vitima* implementa duas políticas. Na LRU (*Least Recently Used*), cada linha possui um contador que é zerado ao ser acessada e incrementado nas demais linhas do mesmo conjunto; a linha com maior contador é substituída. Na política aleatória, a vítima é escolhida por sorteio dentro do conjunto. Em ambos os casos, linhas inválidas têm prioridade.

**Políticas de escrita.** No *write-through*, toda escrita é propagada imediatamente à memória principal. No *write-back*, a escrita ocorre apenas na cache (ativando o bit *dirty*), e o bloco é escrito na memória principal somente quando substituído. A função *finaliza\_simulacao* percorre todas as linhas ao

final e escreve na memória principal aquelas com bit *dirty* ativo (necessário no *write-back*).

**Núcleo da simulação.** A função *acessa* recebe o endereço e a operação, decompõe o endereço, verifica se o bloco está presente na cache (*hit*) e, em caso de *miss*, carrega o bloco da memória principal, substituindo a vítima escolhida pela política ativa. A função *escreve\_resultados* calcula e exibe a taxa de acerto e o AMAT (*Average Memory Access Time*), dado por  $AMAT = t_{hit} + (1 - h) \times t_{mp}$ , onde  $h$  é a taxa de acerto global.

O código-fonte e uma versão executável estão disponíveis no repositório: <https://github.com/phpfc/simulador-cache>.

## 2. Impacto do Tamanho da Cache

Neste experimento, o tamanho do bloco foi fixado em 128 bytes, com política *write-through*, LRU e associatividade de 4 vias. O número de blocos da cache foi variado em potências de 2, de 8 até 1024 blocos, resultando em caches de 1 KB a 128 KB. Os demais parâmetros fixos são apresentados na Tabela 1 do Anexo A; os resultados completos encontram-se na Tabela 2.

A Figura 1 apresenta a taxa de acerto em função do tamanho da cache em bytes.

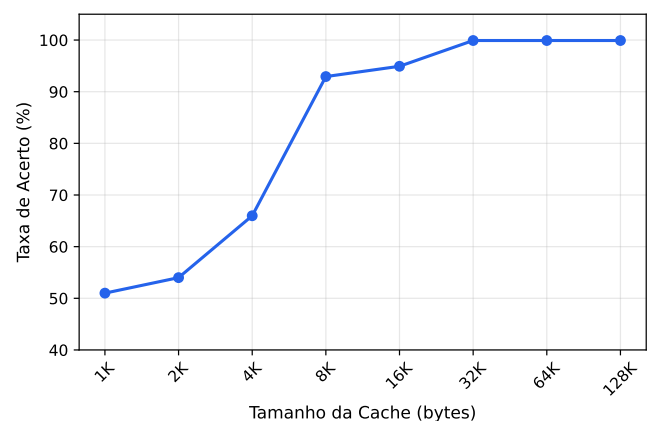


Figura 1: Taxa de acerto em função do tamanho da cache.

O gráfico apresenta um crescimento acentuado da taxa de acerto até aproximadamente 8 KB, seguido de uma estabilização próxima a 99,9% a partir de 32 KB. Essa forma da curva está diretamente relacionada ao princípio da localidade e ao conceito de *working set*. Segundo Tanenbaum e Austin, “a observação de que as referências à memória feitas em qualquer intervalo curto de tempo tendem a usar apenas uma pequena

\*Fundamentos de Arquitetura de Computadores — Universidade de Caxias do Sul (UCS), Caxias do Sul, RS, Brasil.

fração da memória total é chamada de princípio da localidade e constitui a base de todos os sistemas de cache” [1]<sup>83</sup>.

Quando a cache é pequena, ela não comporta o conjunto de trabalho do programa, gerando frequentes substituições de blocos e, consequentemente, uma baixa taxa de acerto. À medida que a cache cresce, uma fração maior do *working set* permanece residente, explorando tanto a localidade temporal quanto a espacial. Porém, uma vez que a cache é suficientemente grande para conter todo o *working set*, aumentos adicionais de tamanho não produzem ganhos significativos, pois praticamente todos os acessos já resultam em acertos. Stallings corrobora essa análise ao afirmar que o tamanho da cache deve ser “grande o suficiente para que o tempo médio de acesso global se aproxime do tempo de acesso da cache” e que “o desempenho da cache é muito sensível à natureza da carga de trabalho, sendo impossível chegar a um único tamanho ‘ótimo’ de cache” [2]<sup>166</sup>.

### 3. Impacto do Tamanho do Bloco

Para avaliar o impacto do tamanho do bloco, a cache foi fixada em 8 KB, com política *write-through*, LRU e associatividade de 2 vias. O tamanho do bloco foi variado de 8 a 4096 bytes em potências de 2. Os parâmetros fixos constam na Tabela 3 e os resultados na Tabela 4 do Anexo A.

A Figura 2 mostra a taxa de acerto em função do tamanho do bloco.

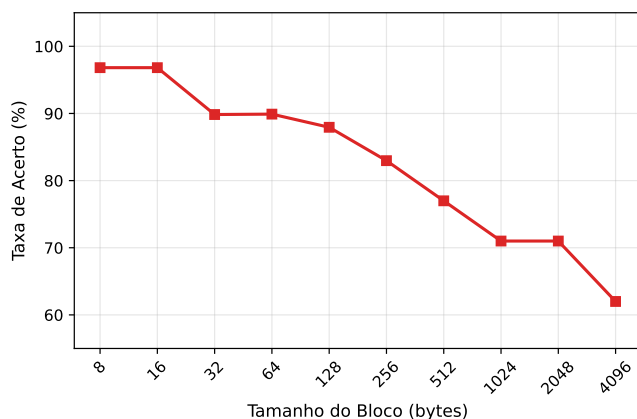


Figura 2: Taxa de acerto em função do tamanho do bloco.

Observa-se que a taxa de acerto é máxima para blocos pequenos (8–16 bytes, em torno de 96,8%) e diminui progressivamente à medida que o bloco aumenta, chegando a 62,0% com blocos de 4 KB. Segundo Stallings, “à medida que o tamanho do bloco aumenta a partir de tamanhos muito pequenos, a taxa de acerto inicialmente cresce devido ao princípio da localidade espacial [...] A taxa de acerto começará a diminuir, contudo, quando o bloco se torna ainda maior”, pois “blocos maiores reduzem o número de blocos que cabem na cache” e “cada palavra adicional está mais distante da palavra requisitada e, portanto, tem menor probabilidade de ser necessária no futuro próximo” [2]<sup>178</sup>.

No caso específico deste experimento, como a cache total é fixa em 8 KB, blocos maiores implicam menos linhas na cache (por exemplo, com blocos de 4 KB restam apenas 2 linhas).

Isso anula a vantagem da localidade espacial, pois as constantes substituições de blocos grandes eliminam dados que seriam reutilizados. O tamanho de bloco ótimo para esta configuração situa-se entre 8 e 16 bytes, onde a localidade espacial é bem explorada sem penalizar excessivamente o número de linhas disponíveis.

### 4. Impacto da Associatividade

Neste experimento, fixaram-se o tamanho do bloco em 128 bytes, a política *write-back*, LRU e o tamanho da cache em 8 KB (64 linhas). A associatividade foi variada de 1 (mapeamento direto) a 64 (totalmente associativo). Os parâmetros fixos constam na Tabela 5 e os resultados na Tabela 6 do Anexo A.

A Figura 3 apresenta a taxa de acerto em função da associatividade.

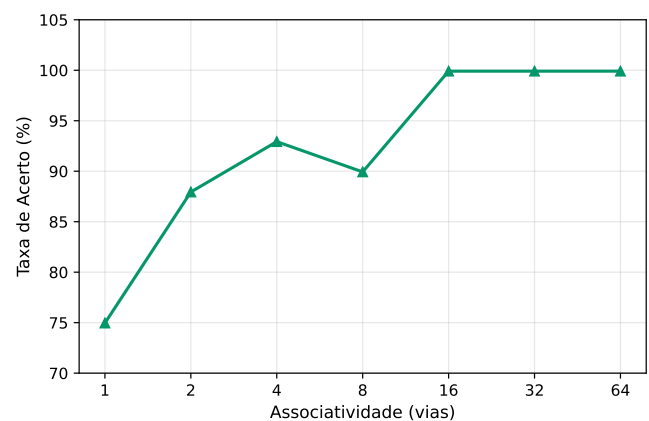


Figura 3: Taxa de acerto em função da associatividade.

A taxa de acerto cresce significativamente de 74,96% (mapeamento direto) para 92,93% (4 vias), apresenta uma queda para 89,93% em 8 vias e então salta para 99,91% a partir de 16 vias, estabilizando-se. A melhoria de mapeamento direto para associativo por conjunto decorre da redução dos *miss* por conflito: no mapeamento direto, cada bloco de memória pode ocupar uma única linha da cache, e blocos que mapeiam na mesma linha provocam substituições mútuas. Com maior associatividade, há mais posições disponíveis em cada conjunto, reduzindo esses conflitos [2]<sup>168</sup>. Conforme observado por Tanenbaum e Austin, “caches de alta associatividade não melhoram significativamente o desempenho em relação a caches de baixa associatividade na maioria das circunstâncias, e em alguns casos apresentam desempenho pior”, sendo rara a utilização de associatividade superior a 4 vias [1]<sup>309</sup>.

### 5. Impacto da Política de Substituição

Fixaram-se o tamanho do bloco em 128 bytes, política *write-through* e associatividade de 4 vias. O número de blocos foi variado de 16 a 1024, comparando as políticas LRU e aleatória. Os parâmetros fixos constam na Tabela 7 e os resultados na Tabela 8 do Anexo A.

A Figura 4 apresenta as duas curvas de taxa de acerto em função do tamanho da cache.

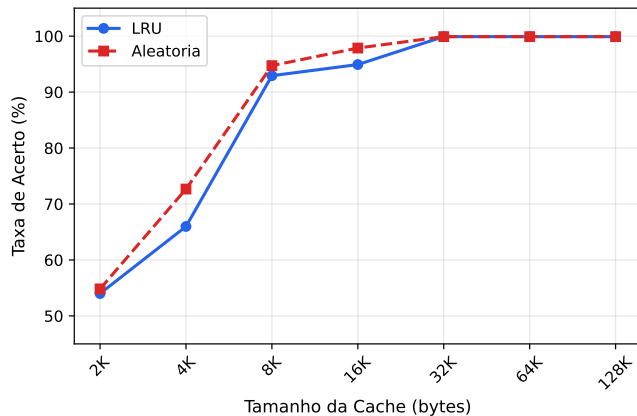


Figura 4: Taxa de acerto em função do tamanho da cache: LRU vs. aleatória.

Observa-se que, para caches pequenas (2 KB a 4 KB), a política aleatória apresenta taxa de acerto ligeiramente superior à LRU. Para caches médias (8 KB a 16 KB), as duas políticas apresentam resultados próximos, com a aleatória mantendo pequena vantagem. A partir de 32 KB, ambas convergem para 99,91%. Stallings afirma que “estudos de simulação têm mostrado que a substituição aleatória proporciona desempenho apenas ligeiramente inferior a um algoritmo baseado em uso” [2]<sup>177</sup>. A convergência para caches grandes é esperada, pois quando a cache comporta todo o *working set*, a política de substituição torna-se irrelevante, já que raramente ocorrem *misses*. A LRU explora a localidade temporal ao manter na cache os blocos acessados mais recentemente [1]<sup>306</sup>, mas seu benefício é mais evidente quando há pressão por espaço na cache.

## 6. Largura de Banda da Memória

Para comparar o tráfego de memória entre *write-through* e *write-back*, foram simuladas caches de 8 KB e 16 KB, com blocos de 64 e 128 bytes, associatividades de 2 e 4 vias, LRU e tempos padrão (hit: 4 ns, MP: 60 ns). Os parâmetros fixos constam na Tabela 9 do Anexo A.

A Tabela 1 apresenta os resultados para ambas as políticas.

Verifica-se que o número de leituras na memória principal é idêntico em ambas as políticas, pois leituras dependem exclusivamente de *misses* de leitura, que são determinados pelo mapeamento e pela associatividade. A diferença concentra-se nas escritas: no *write-through*, toda operação de escrita gera uma escrita na memória principal (6144 em todos os casos, correspondendo ao total de escritas no arquivo de endereços), enquanto no *write-back* ocorrem, em média, apenas 783 escritas na memória principal — uma redução de 87,3%. Portanto, a cache com política *write-back* possui o menor tráfego de memória. Conforme Stallings, no *write-back*, “as atualizações são feitas apenas na cache” e o bloco é “escrito de volta na memória principal se e somente se o bit *dirty* estiver ativo”, minimizando o tráfego no barramento [2]<sup>177</sup>.

## 7. Avaliação Global

**Melhor projeto de cache.** Dentre todas as configurações simuladas, o melhor projeto de cache é aquele que apresenta a maior taxa de acerto aliada ao menor AMAT e ao menor tráfego de memória. A configuração com cache de 32 KB (256 blocos de 128 bytes), associatividade de 4 vias, política *write-back* e LRU atinge taxa de acerto de 99,91% e AMAT de 4,05 ns. Além disso, o *write-back* reduz drasticamente as escritas na memória principal. Essa configuração oferece o melhor equilíbrio entre desempenho, complexidade e tráfego de memória. Conforme discutido por Tanenbaum e Austin, a combinação de uma cache de tamanho adequado ao *working set* com associatividade moderada (4 vias) e política de escrita eficiente resulta em um sistema de memória que se aproxima do tempo de acesso da cache isoladamente [1]<sup>84</sup>.

**Simulação das demais formas de mapeamento.** Sim, é possível simular todas as formas de mapeamento com o simulador desenvolvido. O mapeamento direto é obtido configurando a associatividade igual a 1 (*assoc=1*), de modo que cada conjunto contém apenas uma linha e cada bloco de memória mapeia exatamente uma posição da cache. O mapeamento totalmente associativo é obtido configurando a associatividade igual ao número total de linhas (*assoc=num\_linhas*), resultando em um único conjunto onde qualquer bloco pode ocupar qualquer linha. Essas são os dois casos extremos do mapeamento associativo por conjunto [2]<sup>170</sup>.

**Parte mais difícil do desenvolvimento.** A parte mais desafiadora do desenvolvimento do simulador foi a implementação correta da política de escrita, particularmente a combinação *write-back* com a substituição de linhas sujas (*dirty*). Garantir que o bloco substituído fosse corretamente escrito na memória principal antes de ser sobrescrito, e que a função de finalização atualizasse todos os blocos sujos remanescentes na cache, exigiu atenção especial aos casos de borda e aos fluxos de execução distintos para leitura e escrita.

## Referências

1. TANENBAUM, A. S.; AUSTIN, T. *Structured Computer Organization*. 6. ed. Prentice Hall, 2012.
2. STALLINGS, W. *Computer Organization and Architecture: Designing for Performance*. 11. ed. Pearson, 2019.

Tabela 1: Tráfego de memória: *write-through* vs. *write-back*.

| Cache | Bloco | Assoc. | Pol. | <i>Write-through</i> |         |       | <i>Write-back</i> |         |       |
|-------|-------|--------|------|----------------------|---------|-------|-------------------|---------|-------|
|       |       |        |      | Leit. MP             | Esc. MP | Total | Leit. MP          | Esc. MP | Total |
| 8 KB  | 64 B  | 2      | —    | 5170                 | 6144    | 11314 | 5170              | 1026    | 6196  |
| 8 KB  | 64 B  | 4      | —    | 3126                 | 6144    | 9270  | 3126              | 515     | 3641  |
| 8 KB  | 128 B | 2      | —    | 6176                 | 6144    | 12320 | 6176              | 1026    | 7202  |
| 8 KB  | 128 B | 4      | —    | 3621                 | 6144    | 9765  | 3621              | 515     | 4136  |
| 16 KB | 64 B  | 2      | —    | 2615                 | 6144    | 8759  | 2615              | 1026    | 3641  |
| 16 KB | 64 B  | 4      | —    | 2615                 | 6144    | 8759  | 2615              | 515     | 3130  |
| 16 KB | 128 B | 2      | —    | 3621                 | 6144    | 9765  | 3621              | 1026    | 4647  |
| 16 KB | 128 B | 4      | —    | 2599                 | 6144    | 8743  | 2599              | 515     | 3114  |
| Média |       |        |      | 3693                 | 6144    | 9837  | 3693              | 783     | 4463  |

## Anexo A — Tabelas de Dados

Tabela 2: Parâmetros fixos — Impacto do Tamanho da Cache.

| Parâmetro                 | Valor                |
|---------------------------|----------------------|
| Tamanho do Bloco          | 128 bytes            |
| Política de Escrita       | <i>write-through</i> |
| Algoritmo de Substituição | LRU                  |
| Associatividade           | 4 vias               |
| Tempo de Hit              | 4 ns                 |
| Tempo da MP               | 60 ns                |

Tabela 3: Resultados — Impacto do Tamanho da Cache.

| Núm. Blocos | Taxa de Acerto (%) | AMAT (ns) | Leituras MP | Escritas MP |
|-------------|--------------------|-----------|-------------|-------------|
| 8           | 51,00              | 33,40     | 25088       | 6144        |
| 16          | 54,00              | 31,60     | 23552       | 6144        |
| 32          | 65,98              | 24,41     | 17418       | 6144        |
| 64          | 92,93              | 8,24      | 3621        | 6144        |
| 128         | 94,92              | 7,05      | 2599        | 6144        |
| 256         | 99,91              | 4,05      | 44          | 6144        |
| 512         | 99,91              | 4,05      | 44          | 6144        |
| 1024        | 99,91              | 4,05      | 44          | 6144        |

Tabela 4: Parâmetros fixos — Impacto do Tamanho do Bloco.

| Parâmetro                 | Valor                |
|---------------------------|----------------------|
| Tamanho da Cache          | 8 KB                 |
| Política de Escrita       | <i>write-through</i> |
| Algoritmo de Substituição | LRU                  |
| Associatividade           | 2 vias               |
| Tempo de Hit              | 4 ns                 |
| Tempo da MP               | 60 ns                |

Tabela 5: Resultados — Impacto do Tamanho do Bloco.

| Tam. Bloco (B) | Taxa de Acerto (%) | AMAT (ns) | Leituras MP | Escritas MP |
|----------------|--------------------|-----------|-------------|-------------|
| 8              | 96,82              | 5,91      | 1627        | 6144        |
| 16             | 96,82              | 5,91      | 1626        | 6144        |
| 32             | 89,84              | 10,10     | 5202        | 6144        |
| 64             | 89,90              | 10,06     | 5170        | 6144        |
| 128            | 87,94              | 11,24     | 6176        | 6144        |
| 256            | 82,97              | 14,22     | 8721        | 6144        |
| 512            | 76,99              | 17,81     | 11782       | 6144        |
| 1024           | 71,00              | 21,40     | 14848       | 6144        |
| 2048           | 71,00              | 21,40     | 14848       | 6144        |
| 4096           | 62,00              | 26,80     | 19456       | 6144        |

Tabela 6: Parâmetros fixos — Impacto da Associatividade.

| Parâmetro                 | Valor             |
|---------------------------|-------------------|
| Tamanho do Bloco          | 128 bytes         |
| Tamanho da Cache          | 8 KB              |
| Política de Escrita       | <i>write-back</i> |
| Algoritmo de Substituição | LRU               |
| Tempo de Hit              | 4 ns              |
| Tempo da MP               | 60 ns             |

Tabela 7: Resultados — Impacto da Associatividade.

| Associatividade | Taxa de Acerto (%) | AMAT (ns) | Leituras MP | Escritas MP |
|-----------------|--------------------|-----------|-------------|-------------|
| 1               | 74,96              | 19,03     | 12822       | 2048        |
| 2               | 87,94              | 11,24     | 6176        | 1026        |
| 4               | 92,93              | 8,24      | 3621        | 515         |
| 8               | 89,93              | 10,04     | 5154        | 1026        |
| 16              | 99,91              | 4,05      | 44          | 4           |
| 32              | 99,91              | 4,05      | 44          | 4           |
| 64              | 99,91              | 4,05      | 44          | 4           |

Tabela 8: Parâmetros fixos — Impacto da Política de Substituição.

| Parâmetro           | Valor                |
|---------------------|----------------------|
| Tamanho do Bloco    | 128 bytes            |
| Política de Escrita | <i>write-through</i> |
| Associatividade     | 4 vias               |
| Tempo de Hit        | 4 ns                 |
| Tempo da MP         | 60 ns                |

Tabela 9: Resultados — Impacto da Política de Substituição.

| Núm. Blocos | Política  | Taxa de Acerto (%) | AMAT (ns) | Leituras MP | Escritas MP |
|-------------|-----------|--------------------|-----------|-------------|-------------|
| 16          | LRU       | 54,00              | 31,60     | 23552       | 6144        |
| 16          | Aleatória | 54,83              | 31,10     | 23128       | 6144        |
| 32          | LRU       | 65,98              | 24,41     | 17418       | 6144        |
| 32          | Aleatória | 72,67              | 20,40     | 13993       | 6144        |
| 64          | LRU       | 92,93              | 8,24      | 3621        | 6144        |
| 64          | Aleatória | 94,73              | 7,16      | 2697        | 6144        |
| 128         | LRU       | 94,92              | 7,05      | 2599        | 6144        |
| 128         | Aleatória | 97,87              | 5,28      | 1090        | 6144        |
| 256         | LRU       | 99,91              | 4,05      | 44          | 6144        |
| 256         | Aleatória | 99,91              | 4,05      | 44          | 6144        |
| 512         | LRU       | 99,91              | 4,05      | 44          | 6144        |
| 512         | Aleatória | 99,91              | 4,05      | 44          | 6144        |
| 1024        | LRU       | 99,91              | 4,05      | 44          | 6144        |
| 1024        | Aleatória | 99,91              | 4,05      | 44          | 6144        |

Tabela 10: Parâmetros fixos — Largura de Banda da Memória.

| Parâmetro                 | Valor |
|---------------------------|-------|
| Algoritmo de Substituição | LRU   |
| Tempo de Hit              | 4 ns  |
| Tempo da MP               | 60 ns |

Tabela 11: Resultados — Largura de Banda (*write-through*).

| Cache | Bloco (B) | Assoc. | Taxa (%) | Leituras MP | Escritas MP | Total |
|-------|-----------|--------|----------|-------------|-------------|-------|
| 8 KB  | 64        | 2      | 89,90    | 5170        | 6144        | 11314 |
| 8 KB  | 64        | 4      | 93,89    | 3126        | 6144        | 9270  |
| 8 KB  | 128       | 2      | 87,94    | 6176        | 6144        | 12320 |
| 8 KB  | 128       | 4      | 92,93    | 3621        | 6144        | 9765  |
| 16 KB | 64        | 2      | 94,89    | 2615        | 6144        | 8759  |
| 16 KB | 64        | 4      | 94,89    | 2615        | 6144        | 8759  |
| 16 KB | 128       | 2      | 92,93    | 3621        | 6144        | 9765  |
| 16 KB | 128       | 4      | 94,92    | 2599        | 6144        | 8743  |
| Média |           |        | —        | 3693        | 6144        | 9837  |

Tabela 12: Resultados — Largura de Banda (*write-back*).

| Cache | Bloco (B) | Assoc. | Taxa (%) | Leituras MP | Escritas MP | Total |
|-------|-----------|--------|----------|-------------|-------------|-------|
| 8 KB  | 64        | 2      | 89,90    | 5170        | 1026        | 6196  |
| 8 KB  | 64        | 4      | 93,89    | 3126        | 515         | 3641  |
| 8 KB  | 128       | 2      | 87,94    | 6176        | 1026        | 7202  |
| 8 KB  | 128       | 4      | 92,93    | 3621        | 515         | 4136  |
| 16 KB | 64        | 2      | 94,89    | 2615        | 1026        | 3641  |
| 16 KB | 64        | 4      | 94,89    | 2615        | 515         | 3130  |
| 16 KB | 128       | 2      | 92,93    | 3621        | 1026        | 4647  |
| 16 KB | 128       | 4      | 94,92    | 2599        | 515         | 3114  |
| Média |           |        | —        | 3693        | 783         | 4463  |